

STUDI KASUS TEKNIS — JUNIOR DEVELOPER

Take-home assignment · Stack: Golang atau Java (Spring Boot)

Posisi : Junior Developer

Bentuk : Studi kasus dikerjakan mandiri, dikumpulkan sebagai repository Git

Estimasi waktu kerja : 4–6 jam | **Batas pengumpulan** : 3 hari kalender sejak dokumen diterima

Total nilai : 100 poin (+15 poin bonus)

Batas kelulusan yang disarankan : ≥ 60 poin

Tahap lanjutan : sesi diskusi & penjelasan kode ± 30 menit

Nama peserta : _____ **Tanggal kirim** : _____

1. Aturan Pengerjaan

- Pilih satu track saja** — Track A (Golang) atau Track B (Java/Spring Boot). Mengerjakan keduanya tidak menambah nilai.
- Boleh menggunakan internet, dokumentasi resmi, IDE, library apa pun, dan AI assistant. **Syaratnya: Anda harus mampu menjelaskan setiap baris kode yang dikumpulkan** pada sesi diskusi. Kode yang tidak bisa dijelaskan dianggap tidak dikerjakan.
- Kerjakan sendiri. Dilarang menyalin repository orang lain secara utuh.
- Tidak selesai seluruhnya bukan diskualifikasi.** Kerjakan dari bagian WAJIB lebih dulu, lalu tulis di `README.md` bagian mana yang belum selesai dan bagaimana rencana Anda menyelesaikannya. Kejujuran ini dinilai positif.
- Fokus pada kode yang **benar, rapi, dan teruji** — bukan pada jumlah fitur.
- Bila ada kebutuhan yang menurut Anda ambigu, silakan ambil asumsi sendiri dan **tulis asumsinya di README**.

2. Pilihan Track

Track A — Golang	Track B — Java
Go 1.21+ HTTP: <code>net/http</code> standard library, atau Gin / Echo Database: <code>database/sql</code> + driver, atau GORM Testing: package <code>testing</code> bawaan (<code>go test ./...</code>)	Java 17+ Spring Boot 3 (Spring Web) Database: Spring Data JPA / Hibernate, atau JDBC Build: Maven atau Gradle · Testing: JUnit 5 (+ Mockito bila perlu)

Database: gunakan database relasional — PostgreSQL, MySQL, atau SQLite (SQLite diperbolehkan agar mudah dijalankan pengujian). Sertakan `schema.sql` atau file migrasi. Penyimpanan *in-memory* hanya diperbolehkan bila Anda benar-benar kehabisan waktu, dan harus dicatat di README (ada pengurangan nilai pada kriteria basis data).

STUDI KASUS 1 — Order Service

60 poin

1.1 Latar Belakang

Sebuah toko online skala kecil ingin memisahkan pengelolaan produk dan pemesanan ke dalam sebuah *backend service* tersendiri. Saat ini semua transaksi dicatat manual di spreadsheet, sehingga sering terjadi masalah: stok tercatat minus, total tagihan tidak cocok karena harga produk berubah setelah pesanan dibuat, dan status pesanan bisa berubah tidak beraturan.

Tugas Anda: membangun REST API yang menyelesaikan tiga masalah tersebut.

1.2 Model Data

Rancang tabel berikut (nama kolom boleh disesuaikan, relasi & constraint wajib ada):

```
products      : id, sku (UNIQUE), name, price, stock, created_at
customers     : id, name, email (UNIQUE), created_at
orders        : id, customer_id (FK), status, total_amount, created_at
order_items   : id, order_id (FK), product_id (FK), qty, price_at_order
```

Perhatikan: kolom `price_at_order` pada `order_items` ada dengan sengaja. Pikirkan mengapa harga perlu disimpan ulang di sini, padahal harga sudah ada di tabel `products`. Alasannya akan ditanyakan saat sesi diskusi.

1.3 Endpoint yang Diminta WAJIB

Method	Endpoint	Keterangan
POST	/products	Membuat produk. Validasi: <code>sku</code> & <code>name</code> wajib, <code>price</code> > 0, <code>stock</code> ≥ 0. Sukses → 201 .
GET	/products	Daftar produk dengan pagination (<code>?page=1&limit=10</code>) dan pencarian nama (<code>?q=kaos</code>). Response menyertakan metadata total data.
GET	/products/{id}	Detail produk. Tidak ada → 404 .
PUT	/products/{id}	Ubah <code>name</code> , <code>price</code> , <code>stock</code> .
POST	/customers	Membuat customer. Email harus valid dan unik.
POST	/orders	Membuat pesanan. Ini inti dari studi kasus — lihat aturan bisnis di bawah.
GET	/orders/{id}	Detail pesanan lengkap dengan daftar item (termasuk nama produk dan harga saat pemesanan) serta total.
GET	/orders	Daftar pesanan, bisa difilter <code>?customer_id=</code> dan <code>?status=</code> .
PATCH	/orders/{id}/status	Mengubah status pesanan mengikuti aturan transisi di bawah.

Contoh request pembuatan pesanan

```
POST /orders
{
  "customer_id": 1,
  "items": [
    { "product_id": 10, "qty": 2 },
    { "product_id": 15, "qty": 1 }
  ]
}
```

Perhatikan bahwa client **tidak mengirimkan harga maupun total**. Itu disengaja.

1.4 Aturan Bisnis — bagian yang paling dinilai **WAJIB**

1. **Total dihitung di server.** Harga diambil dari tabel `products` pada saat pesanan dibuat, lalu disalin ke `order_items.price_at_order`. `orders.total_amount` = jumlah (`qty` × `price_at_order`). Harga/total yang dikirim client — bila ada — harus diabaikan.
2. **Stok berkurang** sebesar `qty` untuk setiap item saat pesanan berhasil dibuat.
3. **Stok tidak boleh minus.** Bila stok salah satu produk tidak mencukupi, kembalikan **400** dengan pesan yang jelas (sebutkan produk mana dan sisa stoknya).
4. **Bersifat atomik.** Jika pesanan berisi 3 item dan item ke-3 gagal (stok kurang / produk tidak ada), maka **tidak boleh ada perubahan sama sekali** — tidak ada order tersimpan, dan stok item ke-1 & ke-2 tidak berkurang. Gunakan transaksi database.
5. **Transisi status** hanya boleh mengikuti alur berikut:

```
PENDING → PAID → SHIPPED → COMPLETED
      |           |
      └──> CANCELLED <—
```

Status tidak boleh mundur (mis. `SHIPPED` → `PAID`), dan pesanan yang sudah `COMPLETED` atau `CANCELLED` bersifat final. Transisi tidak valid → **409 Conflict**.

6. **Pembatalan mengembalikan stok.** Saat status berubah menjadi `CANCELLED`, stok seluruh item dikembalikan — juga harus atomik.
7. **Duplikasi data unik** (`sku` produk atau `email` customer yang sudah ada) → **409 Conflict**, bukan 500.
8. **Produk atau customer tidak ditemukan** saat membuat pesanan → **404** atau **400** dengan pesan yang jelas (pilih salah satu, konsisten, dan jelaskan alasannya di README).

1.5 Ketentuan Teknis **WAJIB**

1. **Format response konsisten.** Seluruh response bertipe `application/json`, termasuk error. Tentukan satu format error dan pakai di semua endpoint, contoh:

```
{ "error": "insufficient stock", "detail": "product 'KAOS-01' remaining 3, requested 5" }
```

2. **HTTP status code yang tepat** — 200 / 201 / 204 / 400 / 404 / 409 / 500 sesuai konteks. Jangan mengembalikan 200 untuk semua kondisi.
3. **Pemisahan layer:** minimal `handler/controller` → `service` (aturan bisnis) → `repository` (akses data). Logika bisnis tidak boleh berada di dalam handler.

4. **Konfigurasi lewat environment variable** (koneksi database, port). Jangan di-*hardcode*, jangan commit kredensial ke repository.
5. **Unit test** — minimal **5 skenario**, wajib mencakup:
- pembuatan pesanan berhasil, total terhitung benar;
 - pembuatan pesanan gagal karena stok kurang;
 - stok **tidak berubah** setelah pesanan gagal (menguji atomicity);
 - transisi status tidak valid ditolak;
 - resource tidak ditemukan → 404.
- Test harus benar-benar lulus saat dijalankan (`go test ./... / mvn test`).
6. **Logging** sederhana untuk setiap request (method, path, status, durasi) dan untuk setiap error tingkat server.

1.6 Nilai Tambah BONUS

Dikerjakan **hanya bila bagian wajib sudah selesai**. Lihat rincian poin di Bagian 4.

- **Antarmuka React.js** — satu halaman berisi daftar produk dan form pembuatan pesanan yang memanggil API Anda.
- **CI pipeline** — `.gitlab-ci.yml` atau `Jenkinsfile` yang menjalankan build + test otomatis pada setiap push.
- **Dockerfile** / `docker-compose.yml` sehingga penguji cukup menjalankan satu perintah.
- **Dokumentasi API** — OpenAPI/Swagger atau Postman collection yang disertakan dalam repo.
- **Penanganan race condition** — dua request bersamaan memesan produk terakhir tidak boleh membuat stok minus (kunci baris database, *optimistic locking*, atau constraint `CHECK (stock >= 0)`). Jelaskan pendekatan Anda di README.
- **Graceful shutdown**, middleware autentikasi sederhana (API key/JWT), atau *rate limiting*.